



InfiniFi Protocol

Security Review



Disclaimer

Security Review

InfiniFi Protocol



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

InfiniFi Protocol



Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S--H01	18
3S--H02	20
3S--M01	22
3S--M02	24
3S--M03	26
3S--L01	28
3S--L02	29
3S--L03	30
3S--L04	31
3S--N01	33
3S--N02	35
3S--N03	36
3S--N04	37

Summary Security Review

InfiniFi Protocol



Summary

Three Sigma audited InfiniFi in a 1.2 person week engagement. The audit was conducted from 25/03/2026 to 27/03/2026.

Protocol Description

InfiniFi is a DeFi protocol that enables users to mint and redeem receipt tokens (iUSD, iETH) against collateral assets (USDC, ETH). The protocol features a sophisticated yield generation system through multiple farm integrations, a locking mechanism for enhanced rewards, and a governance system for farm allocation voting.

Scope Security Review

InfiniFi Protocol

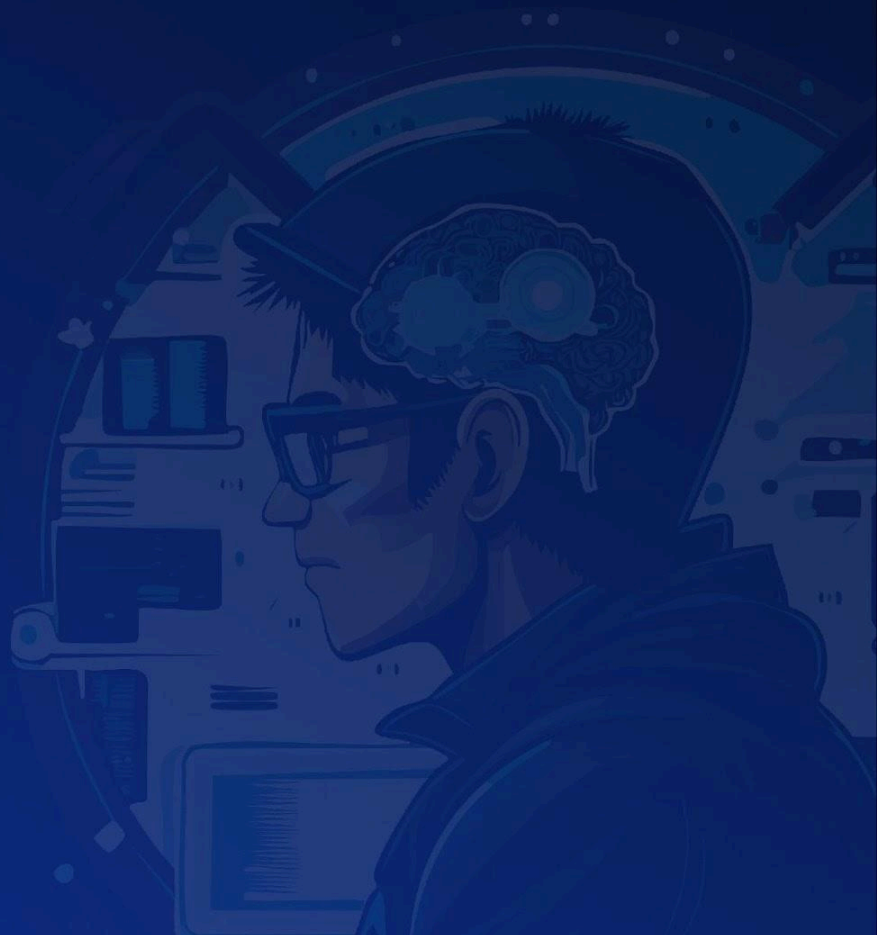


Scope

Filepath	nSLOC
4835527:src/integrations/farms/MorphoUmbrellaFarm.sol	72
F2da46f:src/integrations/farms/CapFarm.sol	49
42ad68f:src/integrations/farms/ParetoFarm.sol	80
68540dd:src/finance/YieldSharingV3.sol	245
68540dd:src/finance/PLSmootherHelper.sol	46
Total	492

Methodology Security Review

InfiniFi Protocol



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard Security Review

InfiniFi Protocol



Project Dashboard

Application Summary

Name	InfiniFi
Repository	https://github.com/InfiniFi-Labs/infinifi-contracts
Commit	PR333; PR336; PR339; PR341
Language	Solidity
Platform	Ethereum

Engagement Summary

Timeline	25/03/2026 to 27/03/2026
Nº of Auditors	2
Review Time	1.2 person weeks

Vulnerability Summary

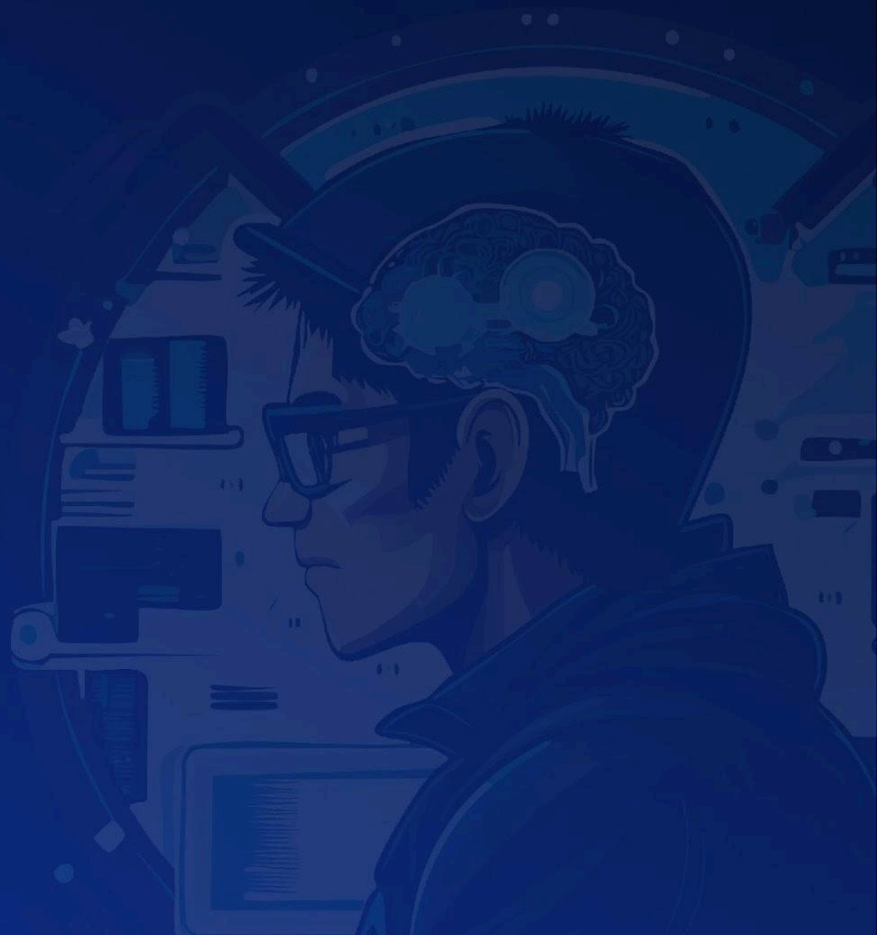
Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	2	2	0
Medium	3	3	0
Low	4	0	4
None	4	3	1

Category Breakdown

Suggestion	4
Documentation	0
Bug	9
Optimization	0
Good Code Practices	0

Risk Section Security Review

InfiniFi Protocol



Risk Section

No risks were identified.

Findings

Security Review

InfiniFi Protocol



Findings

3S-H01

CapFarm::_withdraw misuses **getMintAmount** to derive CUSD burn amounts, causing reverts on saturated mint capacity and incorrect fee accounting

Id	3S--H01
Classification	High
Impact	High
Likelihood	Medium
Category	Bug
Status	Addressed in #f2da46f .

Description

CapFarm integrates with the CUSD protocol by minting CUSD from an underlying asset, staking it in **STCUSD** (an ERC-4626 vault), and reversing this flow on withdrawal. The **_withdraw** function handles two cases: a full withdrawal (when the entire CUSD balance maps exactly to the requested **_amount**) and a partial withdrawal. In the partial withdrawal path, the contract calls **ICapToken(CUSD).getMintAmount(assetToken, _amount + 1)** to determine how many CUSD tokens it should burn to receive exactly **_amount** of the underlying asset.

This approach is flawed for two independent reasons:

1. **Revert on saturated mint capacity.** **Vault::getMintAmount** caps **_amountIn** by **getRemainingMintCapacity(_asset)**. When the deposit cap for the asset is fully utilized, the remaining capacity is zero, so **_amountIn** is reduced to zero and **getMintAmount** returns **amountOut = 0**. The contract then calls **_burn(0, _to)**, which forwards **0** CUSD to **Vault::burn**. Inside **VaultLogic::burn**, the check **if (params.amountOut == 0) revert InvalidAmount()** triggers, causing the entire withdrawal to revert. This makes partial withdrawals completely unusable whenever the CUSD mint capacity for the asset is saturated.
2. **Incorrect amount when not whitelisted.** **getMintAmount** computes the output using mint-direction fee slopes (**minMintFee**, **mintKinkRatio**), while the actual burn performed by **_burn** applies burn-direction fee slopes (**burnKinkRatio**). In

MinterLogic::_applyFeeSlopes, mint fees activate when the post-trade ratio exceeds **optimalRatio**, while burn fees activate when it falls below **optimalRatio**, using different kink parameters. These fee curves are independent and generally asymmetric. The **getMintAmount** result only equals the correct inverse of **getBurnAmount** when the contract is whitelisted in CUSD (both fees become zero). For a non-whitelisted **CapFarm**, the CUSD amount derived from **getMintAmount** does not match the amount needed to receive **_amount** from **burn**, causing the **assert(receiverBalanceIncrease == amount)** check in **Farm::withdraw** to revert or the withdrawal to produce an incorrect asset transfer.

Recommendation

Two approaches can address this issue:

Option 1: Introduce a share-based `redeem` function

Introduce a new **redeem(uint256 stCusdShares, address to)** external function in the **Farm** contract (or directly in **CapFarm**). Instead of targeting an exact asset output amount, this function operates on an exact input of stCUSD shares:

- Redeem the specified stCUSD shares via **ERC4626(STCUSD).redeem(shares, ...)** to receive CUSD.
- Burn the received CUSD via **ICapToken(CUSD).burn(...)** to receive the underlying asset.
- Transfer the resulting asset tokens to the **to** address.

Both **STCUSD** and **CUSD** already expose the necessary **redeem** and **burn** functions that accept exact input amounts, so no inverse calculation is needed. The existing **withdraw** function should be disabled for **CapFarm** (e.g., by overriding it to revert) since it requires exact asset output, which is fundamentally incompatible with the CUSD burn model. The caller (farm manager / rebalancer) would be responsible for computing off-chain how many stCUSD shares to redeem to approximate the desired asset amount.

Option 2: Runtime whitelist check with fee-aware inverse calculation

Query **ICapToken(CUSD).whitelisted(address(this))** at runtime to determine whether the farm is subject to fees. If whitelisted, the relationship between CUSD input and asset output is linear (fee is zero), so a proportional estimate derived from **getBurnAmount** can be used directly to invert the calculation. If not whitelisted, the contract would need to replicate the reverse of **MinterLogic::_applyFeeSlopes** to solve for the CUSD input amount that produces the desired asset output after fees. The required fee parameters (**minMintFee**, **slope0**, **slope1**, **mintKinkRatio**, **burnKinkRatio**, **optimalRatio**) are accessible via **ICapToken(CUSD).getFeeData(assetToken)**, and the oracle prices and supply data needed to compute the post-trade ratio are also publicly available.

3S-H02

Context reuse of **_handlePositiveYield** in **flushExcessSafetyBuffer** **address(0)** path mints unbacked iUSD

Id	3S--H02
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #7d5a639 .

Description

YieldSharingV3::flushExcessSafetyBuffer allows a keeper with the **ACCRUE_YIELD** role to redistribute iUSD tokens that have accumulated in the safety buffer above the configured **safetyBufferSize**. The function supports four recipient paths: **address(0)** (distribute as normal yield), **stakedToken** (send to siUSD holders via escrow), **lockingModule** (send to liUSD holders), or an arbitrary address (direct transfer). The last three paths correctly transfer existing iUSD tokens out of the contract, reducing its balance and consuming the excess. The **address(0)** path instead delegates to **_handlePositiveYield**, which mints new unbacked iUSD equal to the excess amount while leaving the original excess tokens untouched.

_handlePositiveYield was designed to be called exclusively from **accrue()**, where its parameter represents unminted yield backed by real asset growth in the protocol's farms. Its first action (line 272 in **YieldSharingV3.sol**) is to mint **_positiveYield** new iUSD to the contract's own address, bringing **totalSupply** in line with the protocol's asset value. When **flushExcessSafetyBuffer** calls this function with the excess amount, the excess tokens already exist in **totalSupply**. The mint creates a second copy: **totalSupply** increases by **excess**, but the protocol's underlying assets have not changed.

After minting, **_handlePositiveYield** distributes the newly minted tokens to the performance fee recipient, escrow, and locking module. These outbound transfers reduce the contract's balance by exactly the amount minted, returning it to its pre-call value. The original excess tokens remain on the contract, so the excess is never consumed.

This makes the function repeatable. Each call inflates **totalSupply** by **excess** without any corresponding asset backing. With a safety buffer excess of 500 iUSD, ten calls produce 5,000 iUSD of unbacked supply. When **accrue()** is subsequently called, **unaccruedYield()** computes **assetsInReceiptTokens - totalSupply**, which is now deeply negative. This

triggers **_handleNegativeYield**, which follows the loss waterfall: safety buffer absorption, then slashing of liUSD holders, then siUSD holders, and finally a reduction of the iUSD oracle price affecting all holders.

Recommendation

The excess tokens already exist in **totalSupply** and should not be minted again. The simplest correct fix is to burn the excess before delegating to **_handlePositiveYield**, making the operation supply-neutral: the burn removes the existing tokens, and the mint inside **_handlePositiveYield** re-creates them for distribution.

```
if (_recipient == address(0)) {
+   ReceiptToken(receiptToken).burn(excess);
    _handlePositiveYield(excess);
}
```

After the burn, **balanceOf(address(this))** equals **safetyBufferSize** and **totalSupply** decreases by **excess**. The subsequent mint inside **_handlePositiveYield** restores **totalSupply** to its original value, and the distribution logic sends the minted tokens to the correct recipients. The net effect is that **totalSupply** remains unchanged, the excess is consumed, and tokens are distributed through the standard yield split.

3S-M01

Borrower default leaves unclaimable withdrawal requests counted in **assets**, inflating reported balance

Id	3S--M01
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Addressed in #addd6db .

Description

ParetoFarm::assets computes the farm's total value by summing three components: the liquid **assetToken** balance, the pending **vault.withdrawsRequests(address(this))**, and the share-based position valued via **epochVariant.virtualPrice**. The **withdrawsRequests** term represents funds that the farm has requested to withdraw from the Idle CDO and expects to claim via **ParetoFarm::completeWithdraw**.

When a borrower defaults during **IdleCDOEpochVariant::stopEpoch**, the **try this.getFundsFromBorrower(...)** call fails and execution falls into the **catch** block, which invokes **_handleBorrowerDefault**. This function sets **defaulted = true**, pauses the CDO, and disables new withdrawal requests, but it does not increment **epochNumber** and does not clear the pending **withdrawsRequests** mapping in **IdleCreditVault**.

As a result, **ParetoFarm::completeWithdraw** will always revert when it reaches **IdleCreditVault::claimWithdrawRequest**, because the guard **epochNumber <= lastWithdrawRequest[_user]** evaluates to **true** (the epoch was never advanced past the request). The withdrawal requests are permanently locked in a non-claimable state.

Despite this, **ParetoFarm::assets** continues to include the full face value of **vault.withdrawsRequests(address(this))** in its return value. This inflates the reported balance by the entire amount of the locked requests, which can be a significant fraction of the farm's total position. Any system relying on **assets()** for allocation decisions, yield distribution, or NAV calculations will operate on an overstated value until an admin manually intervenes.

Recommendation

Exclude pending withdrawal requests from the **assets** calculation when the Idle CDO has entered a defaulted state. This can be achieved by conditioning the **withdrawsRequests** addition on the CDO not being defaulted.

```
function assets() public view override(Farm, IFarm) returns (uint256) {
    uint256 assetTokenBalance = IERC20(assetToken).balanceOf(address(this));
-   assetTokenBalance += vault.withdrawsRequests(address(this));
+   if (!epochVariant.defaulted()) {
+       assetTokenBalance += vault.withdrawsRequests(address(this));
+   }
    uint256 shareBalance = IERC20(shareToken).balanceOf(address(this));
    uint256 virtualPrice = epochVariant.virtualPrice(shareToken);
    return assetTokenBalance + shareBalance.mulWadDown(virtualPrice);
}
```

3S-M02

ParetoFarm is missing a function to claim instant withdraw requests

Id	3S--M02
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Addressed in #231335a .

Description

The **ParetoFarm** contract integrates with the Pareto protocol (**IdleCDOEpochVariant** and **IdleCreditVault**) to manage deposits and withdrawals across epoch-based lending vaults. The withdrawal lifecycle in Pareto involves two phases: first requesting a withdrawal via **vaultRequestWithdraw**, then claiming the underlying tokens once the request has been fulfilled.

When **ParetoFarm::vaultRequestWithdraw** is called, it invokes **IdleCDOEpochVariant::requestWithdraw**, which can route through two distinct paths depending on the vault's configuration and APR state:

- **Normal withdrawal**: the request is stored in **IdleCreditVault::withdrawsRequests** (or **apr0Users** when APR is zero). These are claimed via **IdleCDOEpochVariant::claimWithdrawRequest**.
- **Instant withdrawal**: when **disableInstantWithdraw** is **false** and the APR has decreased beyond the configured delta, the request is stored in **IdleCreditVault::instantWithdrawsRequests**. These are claimed via **IdleCDOEpochVariant::claimInstantWithdrawRequest**.

The **ParetoFarm** contract only exposes **completeWithdraw**, which calls **epochVariant.claimWithdrawRequest()**. This handles normal withdrawal claims (including the APR-0 case). However, there is no corresponding function to call **epochVariant.claimInstantWithdrawRequest()**. As a result, when a withdrawal is routed through the instant path, the underlying tokens become permanently locked in the **IdleCreditVault** contract with no way for **ParetoFarm** to retrieve them.

Recommendation

Add a function to **ParetoFarm** that claims instant withdraw requests:

```
+ function completeInstantWithdraw() external whenNotPaused checkSlippage  
onlyCoreRole(CoreRoles.FARM_SWAP_CALLER) {  
+   epochVariant.claimInstantWithdrawRequest();  
+ }
```

3S-M03

YieldSharingV3::flushExcessSafetyBuffer does not distribute pending interpolation rewards before modifying the reward distribution state

Id	3S--M03
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #b85068d .

Description

YieldSharingV3::flushExcessSafetyBuffer allows a privileged caller to distribute excess safety buffer funds to various recipients, including proportionally to staked and locking users (**_recipient == address(0)**), directly to staked token holders (**_recipient == stakedToken**), or directly to locking users (**_recipient == lockingModule**). This function operates independently of **accrue**, which is the primary yield distribution entry point.

The **accrue** function calls **distributeInterpolationRewards** at its start to settle any vested staking rewards before computing new yield. However, **flushExcessSafetyBuffer** does not invoke **accrue** before executing its logic. This leads to multiple issues depending on the **_recipient** parameter:

- When **_recipient == address(0)**, the function calls **_handlePositiveYield**, which splits excess rewards between staked and locking users based on their current receipt token balances. Because **accrue** has not run, pending vested rewards have not been distributed to the staked token vault via **distributeInterpolationRewards**, and pending locking rewards have not been deposited via **LockingController::depositRewards**. The result is a stale ratio between **stakedReceiptTokens** and **lockingReceiptTokens**. Additionally, **_handlePositiveYield** overwrites **point.rate** and **point.lastAccrued** when distributing the staking portion, so any rewards that have vested since **max(point.lastAccrued, point.lastClaimed)** but have not yet been distributed are silently absorbed into the new interpolation schedule, disrupting their intended vesting timeline.
- When **_recipient == stakedToken**, the function transfers the excess to the escrow and directly overwrites **point.rate** and **point.lastAccrued**. Any rewards that have vested since **max(point.lastAccrued, point.lastClaimed)** but have not yet been distributed are silently

absorbed into the new interpolation schedule. Those rewards will eventually be distributed, but their intended vesting timeline is disrupted.

Recommendation

Call **accrue** at the beginning of **flushExcessSafetyBuffer** to settle any pending vested rewards and update staked and locking balances with the latest yield distribution before computing any proportional splits. This requires changing **accrue** from **external** to **public** so it can be invoked internally.

3S-L01

Default **maxSlippage** in **CapFarm** is incompatible with Cap protocol's mint fee, causing deposits to always revert

Id	3S--L01
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Acknowledged with note: <i>"We have an agreement with Cap that this mint fee will be 0."</i>

Description

CapFarm is a yield integration that deposits assets into the Cap protocol by minting **cUSD** via **ICapToken::mint** and staking the resulting tokens in the **stcUSD** ERC-4626 vault. The **Farm** base contract enforces a slippage check on every deposit to ensure the farm does not lose more value than the configured tolerance.

CapFarm inherits the default **maxSlippage** of **0.999999e18** from the **Farm** constructor, which requires at least 99.9999% of the deposited amount to be reflected in **assets()** after the operation. However, Cap protocol charges a 0.1% mint fee on USDC deposits on-chain, meaning only ~99.9% of the input value is preserved after minting. Because the fee (0.1%) far exceeds the tolerated slippage (0.0001%), the slippage check in **Farm::deposit** will always revert with **SlippageTooHigh**, rendering **CapFarm** unable to accept any deposits until an admin manually calls **setMaxSlippage**.

Recommendation

Override the default **maxSlippage** in the **CapFarm** constructor to accommodate Cap protocol's known mint and burn fees:

```
constructor(address _core, address _assetToken) Farm(_core, _assetToken) {
+   maxSlippage = 0.998e18;
}
```


3S-L02

Missing **isAssetSupported** check in

CapFarm::withdrawSecondaryAsset allows withdrawal of arbitrary tokens

Id	3S--L02
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Acknowledged with note: <i>“As this would be used by manual rebalancer which has a builtin check if the asset is supported, this would be redundant. So in order to avoid a double check, I removed it.”</i>

Description

CapFarm::withdrawSecondaryAsset is intended to let the **FARM_MANAGER** withdraw secondary assets (specifically staked Cap tokens, **STCUSD**) from the farm. The contract defines **isAssetSupported** at line 26, which returns **true** only for the **STCUSD** address, establishing a clear allowlist of valid secondary assets.

However, **withdrawSecondaryAsset** never calls **isAssetSupported** to validate the **_asset** parameter. Any address with the **FARM_MANAGER** role can pass an arbitrary token address and transfer it out of the contract. This bypasses the intended restriction and is inconsistent with the pattern used in **MultiAssetFarm::withdrawSecondaryAsset**, which explicitly requires **isAssetSupported(_asset)** before proceeding.

Recommendation

Validate **_asset** via **isAssetSupported** at the top of **CapFarm::withdrawSecondaryAsset**.

3S-L03

Missing array length validation in **vaultDeposit** and **vaultRedeem**

Id	3S--L03
Classification	Low
Impact	Low
Likelihood	Low
Category	Bug
Status	Acknowledged with note: <i>"It will be run by a keeper."</i>

Description

MorphoUmbrellaFarm::vaultDeposit and **MorphoUmbrellaFarm::vaultRedeem** accept two parallel arrays, **_vaults** and **_amounts**, that are iterated together to deposit into or redeem from multiple Morpho ERC-4626 vaults in a single call. The loop iterates over **_vaults.length** and indexes into **_amounts** at the same position, but neither function verifies that the two arrays have the same length.

When **_amounts** is shorter than **_vaults**, the transaction will revert with an out-of-bounds access, which is a safe but uninformative failure. When **_amounts** is longer than **_vaults**, the extra entries in **_amounts** are silently ignored. In that scenario the caller (a **FARM_SWAP_CALLER** role) believes it has deposited or redeemed a set of amounts across vaults, but some amounts were never processed. Because **syncAssets** is called at the end of each function, the cached total assets will reflect the actual on-chain state, yet the caller's off-chain accounting may diverge from it, potentially leading to incorrect rebalancing decisions.

Recommendation

Enforce that **_vaults.length == _amounts.length** at the top of both **MorphoUmbrellaFarm::vaultDeposit** and **MorphoUmbrellaFarm::vaultRedeem**.

3S-L04

Stale yield distribution when protocol parameters are updated without prior accrual

Id	3S--L04
Classification	Low
Impact	Medium
Likelihood	Low
Category	Bug
Status	Acknowledged

Description

The **YieldSharingV3::accrue** function is responsible for distributing protocol yield between two categories of users: staked users (siUSD holders) and locking users (liUSD holders). The split ratio is computed in **YieldSharingV3::_handlePositiveYield** using several configurable parameters:

- **liquidReturnMultiplier** - scales the weight of staked receipt tokens
- **targetIlliquidRatio** - sets a minimum floor for the locking users' share of rewards
- **LockingController::rewardMultiplier()** - a derived value that depends on per-bucket multipliers configured via **LockingController::setBucketMultiplier**

These parameters are independently updatable by governance through **YieldSharingV3::setLiquidReturnMultiplier**, **YieldSharingV3::setTargetIlliquidRatio**, and **LockingController::setBucketMultiplier**, none of which invoke **accrue** before writing the new value.

Because **accrue** is not called before these updates, any yield that has accumulated since the last accrual will be retroactively distributed using the newly set parameters rather than the parameters that were in effect while the yield was generated. This results in an incorrect distribution of rewards: one user group receives more than it earned, while the other receives less.

Recommendation

Ensure that any governance script or multisig transaction that updates **liquidReturnMultiplier**, **targetIlliquidRatio**, or bucket multipliers in **LockingController** always calls **YieldSharingV3::accrue** before applying the parameter change. Document

this requirement clearly in the deployment and operational scripts so that future parameter updates do not inadvertently redistribute previously accumulated yield under incorrect ratios.

3S-N01

ParetoFarm::vaultRequestWithdraw with zero amount unexpectedly withdraws the entire position

Id	3S--N01
Classification	None
Category	Suggestion
Status	Addressed in #4fe7bdd .

Description

ParetoFarm::vaultRequestWithdraw allows a caller with the **FARM_SWAP_CALLER** role to request a withdrawal of a specified amount of tranche tokens from the Pareto vault by calling **IdleCDOEpochVariant::requestWithdraw**.

In **IdleCDOEpochVariant::requestWithdraw**, passing **_amount == 0** is treated as a convenience shorthand meaning "withdraw the caller's entire tranche token balance." When the instant withdrawal path is taken, the function recalculates **_underlyings** from the full balance and sets **_amount = _userTrancheTokens**. The same logic applies on the normal withdrawal path, where **_amount** and **_underlyings** are both recomputed from the caller's full tranche balance. In both cases, **_withdrawOps** subsequently burns all of **ParetoFarm's** tranche tokens and decreases the NAV accordingly.

However, **ParetoFarm::vaultRequestWithdraw** does not validate that **_amount** is non-zero before forwarding the call. A **FARM_SWAP_CALLER** passing **_amount = 0** (whether by mistake, a miscalculation upstream, or misunderstanding the parameter semantics) would unintentionally trigger a full withdrawal of the farm's entire tranche position rather than a no-op. This contradicts the typical convention where a zero amount results in no action.

Recommendation

Either add a zero-amount check to prevent unintentional full withdrawals, or document the behavior so callers are aware of the semantics:

Option A: Add a validation check

```
function vaultRequestWithdraw(uint256 _amount)
    external
    whenNotPaused
```

```

    checkSlippage
    onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
{
+   require(_amount > 0, "Amount must be > 0");
    IERC20(shareToken).forceApprove(address(epochVariant), _amount);
    epochVariant.requestWithdraw(_amount, shareToken);
}

```

Option B: Document the edge case

```

+/// @notice Request a withdrawal of tranche tokens from the Pareto vault.
+/// @param _amount Amount of tranche tokens to withdraw. NOTE: passing 0 will
request
+/// a withdrawal of the farm's entire tranche token balance, as IdleCDOEpochVariant
+/// interprets zero as "withdraw all."
function vaultRequestWithdraw(uint256 _amount)
    external
    whenNotPaused
    checkSlippage
    onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
{
    IERC20(shareToken).forceApprove(address(epochVariant), _amount);
    epochVariant.requestWithdraw(_amount, shareToken);
}

```

3S-N02

Unnecessary share token approval in

ParetoFarm::vaultRequestWithdraw

Id	3S--N02
Classification	None
Category	Suggestion
Status	Addressed in #6315c4a .

Description

ParetoFarm::vaultRequestWithdraw allows authorized callers to request a withdrawal from the Pareto epoch variant vault by specifying an amount of tranche (share) tokens to redeem. Before calling **IdleCDOEpochVariant::requestWithdraw**, the function grants an ERC-20 approval of the share token to the epoch variant contract.

However, this approval is never consumed. **IdleCDOEpochVariant::requestWithdraw** does not call **transferFrom** on the tranche token. Instead, it delegates to **_withdrawOps**, which calls **IdleCDOTranche::burn(msg.sender, _amount)**, burning the tranche tokens directly from the caller's balance without requiring any prior allowance. The approval therefore has no effect and leaves a stale allowance on the share token.

Recommendation

Remove the unnecessary **forceApprove** call in **ParetoFarm::vaultRequestWithdraw**.

```
function vaultRequestWithdraw(uint256 _amount)
    external
    whenNotPaused
    checkSlippage
    onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
{
-   IERC20(shareToken).forceApprove(address(epochVariant), _amount);
    epochVariant.requestWithdraw(_amount, shareToken);
}
```

3S-N03

Incorrect NatSpec on **flushExcessSafetyBuffer**

Id	3S--N03
Classification	None
Category	Suggestion
Status	Addressed in #4f1dbbb .

Description

YieldSharingV3::flushExcessSafetyBuffer allows a privileged caller with the **ACCRUE_YIELD** role to redistribute any excess held above the **safetyBufferSize**. The NatSpec on line 378 states that "yield distributed through this function is ignoring the performance fee." However, when the **_recipient** parameter is **address(0)**, the function delegates directly to **_handlePositiveYield(excess)** at line 393. Inside **_handlePositiveYield**, lines 291-298 unconditionally deduct the **performanceFee** percentage from the yield amount and transfer the fee to **performanceFeeRecipient** before splitting the remainder between stakers and lockers.

This means the **address(0)** path does charge the performance fee, contradicting the NatSpec guarantee. Any integrator or governance participant relying on the documented behavior would incorrectly assume that flushing via **address(0)** distributes the full excess amount as yield. In contrast, the **stakedToken**, **lockingModule**, and direct-transfer paths all bypass **_handlePositiveYield** and therefore genuinely skip the fee, making the inconsistency specific to **address(0)**.

Recommendation

Either update the NatSpec to accurately reflect that the **address(0)** path does charge the performance fee, or modify the **address(0)** code path to bypass the fee and align with the documented intent (e.g. by adding a **bool chargePerformanceFee** parameter to **_handlePositiveYield**).

3S-N04

Off-by-one boundary in **PLSmoother::vestingPerSecond** misaligns with actual vesting window

Id	3S--N04
Classification	None
Category	Suggestion
Status	Acknowledged

Description

The **PLSmoother** contract smooths yield distributions by vesting profit linearly over a configured duration. The **vestingPerSecond** function is a view that reports the aggregate rate of tokens currently vesting per second across all active smoothing items. External consumers rely on this value to gauge the current rate of yield distribution.

The **vested** function determines the amount of tokens that have actually vested for each smoothing item using **elapsed = block.timestamp - data.startTimestamp**. Under this formula:

- At **block.timestamp == data.startTimestamp**, **elapsed** is 0, so **itemVested** is 0 (no vesting has occurred yet).

- At **block.timestamp == data.startTimestamp + data.duration**, **elapsed == data.duration**, so **itemVested** equals the full **data.amount** (the final portion of vesting completes).

This means the effective vesting window is **(data.startTimestamp, data.startTimestamp + data.duration]**.

However, the boundary check in **vestingPerSecond** uses:

```
if (block.timestamp >= data.startTimestamp && block.timestamp < data.startTimestamp + data.duration)
```

This corresponds to the range **[data.startTimestamp, data.startTimestamp + data.duration)**, which is shifted one second earlier compared to the actual vesting window. As a result, the reported rate includes an item at **data.startTimestamp** when no vesting is occurring, and excludes it at **data.startTimestamp + data.duration** when the final second of vesting is still active.

Recommendation

Adjust the boundary in **vestingPerSecond** to match the vesting window defined by **vested**:

```
-      if (block.timestamp >= data.startTimestamp && block.timestamp <
data.startTimestamp + data.duration) {
+      if (block.timestamp > data.startTimestamp && block.timestamp <=
data.startTimestamp + data.duration) {
```